

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a63b5d37bc87c06ff.jsonl`  
- SHA-256: `4e529130331940a82bb375fdb16e7790dabdf9a9a7e37e4aaf8991c445ed77e4`  
- Source modified: `2026-07-06T19:19:59+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T19:18:37.648Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).  
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.  
No code changes. Read the ADR:  
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:  
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').  
Relevant current code (on main): src/tg\_video\_filter/models.py (FilterConfig, ContentItem, Source, Policy),  
src/tg\_video\_filter/db.py (load\_filter\_config/save\_filter\_config shims, sources.watched gate should\_ingest\_source, delivery\_attempts/record\_delivery), src/tg\_video\_filter/telethon\_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery\_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content\_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
  - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,  
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).  
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).  
FINDING (cross\_adr): Tier 2 and Tier 3 caption-text stages depend on data the pipeline is never given: captions are dropped at ingestion and PolicyContext has no VideoMeta  
severity high | section Topic preferences ? Tier 2 ("message caption keyword match"); Security ? Tier 3 ("caption text scanning", "URL/domain reputation... links in captions")  
summary: Tier 2 caption keyword match and Tier 3 caption URL/domain scanning both require the message caption. But the live handler never persists caption text: it builds a VideoRecord (no text field) which insert\_video maps to a ContentItem with text left None. content\_item.text is therefore always NULL for freshly-ingested items, and PolicyContext (content\_item, source, policy) exposes no live Telethon message. Both stages have no input to operate on.  
why\_it\_matters: F2 (caption keyword) and F4 (caption link extraction) are specified against content\_item.text, which is guaranteed empty on the ingestion path. As designed the stages silently match nothing. This is a real gap that would make F2/F4 ship broken; the ADR should call out that ingestion must first be extended to capture caption text (a new dependency/precondition, likely a VideoMeta field + insert\_video change) before these stages are viable.  
evidence: VideoMeta (filters/base.py:15) has NO text/caption field ? only duration/dims/size/mime/is\_gif/is\_round/nosound. Handler builds VideoRecord (telethon\_client.py:262) with no text. insert\_video constructs ContentItem(content\_type='video', ...) and never sets text (db.py insert\_video; text is only ever READ, at db.py:865, in a projection). ContentItem.text default is None (models.py). ADR 0011 itself lists content\_items.text as a "dead-on-arrival column... NULL until later tracks populate them." Nothing in Track B/C/E populates it.  
rec: Add an explicit precondition/dependency: "Capturing caption text at ingestion (VideoMeta.text or event.message.raw\_text -> ContentItem.text) is a prerequisite for F2/F4 and is not yet implemented." Either fold that into F1/F2 scope or add an F0 ingestion-enrichment phase. Do not describe content\_item.text as an available PolicyContext input without this.

## 2. assistant (2026-07-06T19:18:39.916Z)

I'll verify this finding by reading the ADR and checking the codebase claims.

## 3. user (2026-07-06T19:18:41.445Z)

# ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

## Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

## What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

## What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

## Decision

### Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)           ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

### Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem    # from ADR 0011
    source: Source               # the source it came from
    policy: Policy               # the owning policy config

@dataclass
class StageResult:
```

```

    passed: bool
    reason: str # human-readable explanation
    stage_name: str # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### **4. assistant (2026-07-06T19:18:43.320Z)**

Now let me verify the key claims about the code.

### **5. user (2026-07-06T19:18:46.046Z)**

```
main
---VideoMeta---
15:class VideoMeta(BaseModel):
16-     """Metadata extracted from a Telegram video message.
```

```

17-
18- All fields are optional because Telegram metadata can be partial; filters
19- decide how to treat ``None`` (by default: treat as a non-match with a
20- descriptive reason).
21- """
22-
23- duration_s: int | None = None
24- width: int | None = None
25- height: int | None = None
26- size_bytes: int | None = None
27- mime_type: str | None = None
28- is_gif: bool | None = None
29- is_round: bool | None = None
30- nosound: bool | None = None
31-
32-
33-@dataclass(frozen=True)
34-class FilterResult:
35-    """Outcome of evaluating one (or many) filter(s) against a ``VideoMeta``.

```

## 6. user (2026-07-06T19:18:46.743Z)

```

---ContentItem---
149:class ContentItem(BaseModel):
150-    """A normalized content record discovered from any :class:`Source` (ADR 0011).
151-
152-    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153-    (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154-    NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155-    ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156-    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157-    """
158-
159-    model_config = ConfigDict(frozen=True)
160-
161-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162-    account_id: int = Field(description="FK ? accounts.id")
163-    source_id: int = Field(description="FK ? sources.id")
164-    provider: str = Field(default="telegram", description="Provider namespace")
165-    provider_message_id: str = Field(
166-        description="Provider-native message id (Telegram message_id, ?)"
167-    )
168-    content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
169-    text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170-    link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
171-    media_ref: str | None = Field(default=None, description="Provider media reference")
172-    document_id: str | None = Field(
173-        default=None,
174-        description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175-    )
176-    duration_s: int | None = None
177-    width: int | None = None
178-    height: int | None = None
179-    size_bytes: int | None = None

```

## 7. assistant (2026-07-06T19:18:50.766Z)

The finding says the branch is `main` ? actually I'm on main, but the finding claims are about main-state code, which is correct. `VideoMeta` has no text field, `ContentItem.text` defaults to None. Now let me check the ingestion path: VideoRecord construction and insert\_video.

## 8. user (2026-07-06T19:18:52.901Z)

```

38:from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
163:     4. inserts a VideoRecord row,
258:         record = VideoRecord(

```

## 9. user (2026-07-06T19:18:53.695Z)

```

383:class VideoRecord(BaseModel):
384-    """A seen video message + its metadata. Drives dedup and future /search.
385-
386-    Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
387-    New code should use :class:`ContentItem`; the ``db`` layer maps
388-    ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
389-    """
390-
391-    model_config = ConfigDict(frozen=True)
392-
393-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
394-    user_id: int
395-    channel_id: int = Field(description="Origin Telegram channel id")
396-    message_id: int = Field(description="Telegram message id within the channel")
397-    document_id: int | None = Field(
398-        default=None,
399-        description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
400-    )
401-
402-    # Metadata extracted from the Telethon message (may be None if absent).
403-    duration_s: int | None = None
404-    width: int | None = None
405-    height: int | None = None
406-    size_bytes: int | None = None
407-    mime_type: str | None = None
408-
409-    matched: bool = False
410-    forwarded: bool = False
411-    seen_at: datetime = Field(default_factory=_utcnow)
412-
413-    @property
414-    def dedup_key(self) -> tuple[int, int, int]:
415-        """The natural key used for deduplication in the ``videos`` table."""
416-        return (self.user_id, self.channel_id, self.message_id)
417-
418-    @property
419-    def content_fingerprint(self) -> tuple[int, int, int, int] | None:
420-        """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).
421-
422-        Returns None if any field is missing (cannot fingerprint).
423-        """

```

## 10. assistant (2026-07-06T19:18:56.096Z)

VideoRecord has no text/caption field. The handler builds VideoRecord at line 258. Let me confirm the VideoRecord construction and the insert\_video mapping.

## 11. user (2026-07-06T19:18:57.787Z)

```

        Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
    )
except Exception:
    log.warning(
        "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
    )

result = engine_holder[ENGINE_KEY].evaluate(meta)
record = VideoRecord(

```



```

        user_id=user.id,
        channel_id=channel_id,
        message_id=message_id,
        document_id=document_id,
        duration_s=meta.duration_s,
        width=meta.width,
        height=meta.height,
        size_bytes=meta.size_bytes,
        mime_type=meta.mime_type,
        matched=result.matched,
    )
    try:
        inserted = await db.insert_video(conn, record)
    except sqlite3.IntegrityError:
        # Lost a dedup race: a concurrent delivery of the same file
        # (same document_id or same message instance) won the insert.
        # The three-tier check above already classified this as new, so
        # the collision means another task is processing (or has
        # processed) it ? log and skip.
        log.info(
            "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
            user.id,
            channel_id,
            message_id,
            document_id,
        )
    return

```

## 12. user (2026-07-06T19:18:58.919Z)

```

858:     return ContentItem(
865:         text=row["text"],
1601:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1620:     item = ContentItem(

```

## 13. user (2026-07-06T19:19:03.699Z)

```

async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Deprecated shim: insert into ``content_items`` (ADR 0011).

```

Resolves the legacy ``channel\_id`` to a :class:`Source` row (upserting it if absent), computes the provider-neutral ``dedup\_key`` via :func:`compute\_dedup\_key`, and inserts a :class:`ContentItem`. The legacy ``VideoRecord`` shape is reconstructed from the inserted row.

"""

```

src = await upsert_source(
    conn,
    Source(
        account_id=video.user_id,
        provider="telegram",
        provider_source_id=str(video.channel_id),
        kind="channel",
        watched=False,
    ),
)
assert src.id is not None # upsert_source populates the surrogate PK
item = ContentItem(
    account_id=video.user_id,
    source_id=src.id,
    provider="telegram",
    provider_message_id=str(video.message_id),
    content_type="video",
    document_id=str(video.document_id) if video.document_id is not None else None,
    duration_s=video.duration_s,
    width=video.width,

```

```

height=video.height,
size_bytes=video.size_bytes,
mime_type=video.mime_type,
dedup_key=compute_dedup_key(
    provider="telegram",
    document_id=(str(video.document_id) if video.document_id is not None else None),
    source_id=src.id,
    provider_message_id=str(video.message_id),
    size_bytes=video.size_bytes,
    duration_s=video.duration_s,
    height=video.height,
    width=video.width,
),
matched=video.matched,
seen_at=video.seen_at,
)
item = await insert_content_item(conn, item)
return _content_item_to_video(item, src, video.user_id, video.channel_id)

```

```

async def mark_forwarded(conn: aiomysqlite.Connection, video_id: int) -> None:
    """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).

```

```

Per-destination delivery state moves to ``delivery_attempts`` (Track C,
#28). For Phase 1 (before Track C lands) there is no equivalent column on
``content_items`` ? the ``matched`` flag alone is sufficient because v1
delivery still targets the single ``delivery_target`` and retried delivery
is harmless (forwards are idempotent). This shim is retained so existing
call sites in ``delivery.py`` keep working without a flag day.
"""

```

```

# content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
# The row's ``matched`` flag (set at insert time) is the surviving signal.

```

#### 14. assistant (2026-07-06T19:19:08.508Z)

Confirmed: `insert\_video` constructs `ContentItem` with no `text=` argument, so `text` stays at its default `None`. The ingestion path never captures caption text. Let me verify the ADR 0011 claim about `text` being a "dead-on-arrival" NULL column and confirm the event.message provides raw\_text (to validate the recommendation is feasible).

#### 15. user (2026-07-06T19:19:11.788Z)

```

10:## Context
55:content_items.dedup_key TEXT NOT NULL
139:     name                TEXT NOT NULL,                -- e.g. "default", "reggae-route"
140:     config_json           TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
142:     created_at            TEXT NOT NULL,
143:     updated_at            TEXT NOT NULL,
198:     provider              TEXT NOT NULL DEFAULT 'telegram',
199:     provider_source_id     TEXT NOT NULL,                -- Telegram channel_id; future: WA group id
200:     title                  TEXT,
201:     kind                   TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
203:     seen_first_at         TEXT NOT NULL,
204:     last_seen_at          TEXT,
225:     provider              TEXT NOT NULL DEFAULT 'telegram',
226:     provider_message_id    TEXT NOT NULL,                -- Telegram message_id; future: WA message id
227:     content_type           TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' | ...
228:     text                   TEXT,                        -- caption / post text (nullable)
229:     link_url               TEXT,                        -- for link/article posts
230:     media_ref              TEXT,                        -- provider media reference (file path, blob
id)
231:     document_id           TEXT,                        -- Telegram document.id (kept for TG-specific
lookups)
236:     mime_type             TEXT,

```

#### 16. user (2026-07-06T19:19:12.723Z)

```
168:         message: Message = event.message # type: ignore[assignment]
```

#### 17. assistant (2026-07-06T19:19:16.732Z)

The handler has ``event.message`` (a Telethon Message) available at line 168 ? which carries ``raw_text`/caption` ? but never reads it into the record. This confirms the data is present at ingestion but dropped. Let me verify the ADR 0011 "dead-on-arrival" phrasing and check whether any Track B/C/E code populates ``text``.

#### 18. user (2026-07-06T19:19:18.987Z)

```
377:- Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
402:3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
---populate text anywhere---
865:         text=row["text"],
```

#### 19. user (2026-07-06T19:19:22.230Z)

- The dedup-key derivation logic is in code, not the schema ? needs its own unit tests (a ``compute_dedup_key(provider, **metadata)`` pure function).
- Phase 1 carries some dead-on-arrival columns (``content_items.link_url``, ``text``) that aren't populated until later tracks. This is intentional (avoid future migrations) but means some columns are NULL until then.

### Alternatives considered

#### 20. assistant (2026-07-06T19:19:33.508Z)

All claims in the finding are verified. Let me now check the ADR text itself to confirm it does describe ``content_item.text`` as an available input and doesn't already flag this precondition.

The ADR states in the Tier 2 section: "message caption keyword match" (metadata-only in v1); F2 scope is "source title/caption keyword matching"; F4 is "caption link extraction". The ``PolicyContext`` dataclass exposes only ``content_item``, ``source``, ``policy`` ? no live Telethon message. The References section says "ContentItem as the PolicyContext input". Nowhere does the ADR flag that caption text isn't captured at ingestion.

Let me double check the ADR doesn't mention any ingestion-enrichment precondition.

#### 21. user (2026-07-06T19:19:35.955Z)

```
60:?         caption text scanning                                     ?
116:message caption keyword match, and (future) channel topic tags from
127:2. **URL/domain reputation**: check links in captions against a
186:| **F2** ? Topic preferences | `topics` config parsing, source title/caption keyword
matching | F1 |
188:| **F4** ? URL/domain reputation | Domain blocklist, caption link extraction | F1 |
```

#### 22. assistant (2026-07-06T19:19:48.194Z)

Confirmed. The ADR references caption/caption-text in five places (Tier 2, Tier 3, F2, F4) and never once flags that caption text is not captured at ingestion, not populated on ``content_item.text``, and not exposed anywhere in ``PolicyContext``. The ADR even points to ADR 0011 for ``ContentItem`` as the ``PolicyContext`` input ? and ADR 0011 explicitly calls ``text`` a dead-on-arrival NULL column.

The finding holds against the ADR. Let me assess severity calibration.

The finding is a real gap: F2 and F4 are specified against caption text that is guaranteed empty on the ingestion path. As designed, these stages would ship matching nothing. However, since this is a Proposed draft design doc:

- It is not data loss (nothing existing is destroyed).
- The fix is feasible ? ``event.message.raw_text`` is available at the handler (line 168), so it's a straightforward ingestion enrichment (add a field to VideoMeta/VideoRecord and thread it through ``insert_video``).
- But it is a genuine missing precondition/dependency that would make two of the seven phases (F2, F4) ship broken if implemented literally against the design contract as written. That is exactly the "design gap that would break an F-phase" severity band the prompt calls high.

The finding is accurate and its recommendation (add an explicit ingestion-enrichment precondition, e.g. an F0, and stop describing ``content_item.text`` as an available input) is sound. It's not merely a wording nit ? it's a missing dependency that invalidates the input contract for two phases.

I'll confirm at high severity: it's a load-bearing inaccuracy (the design presents ``content_item.text`` as usable input when it is structurally always NULL) that breaks F2/F4 as designed.

**23. user (2026-07-06T19:19:58.937Z)**

Structured output provided successfully